CS6008 Network Security, Assignment 2: DNS Security

Report

Shiv Narang (23B1075) and Sabil Ahmed (23B1057)

October 2026

Overview

We build a small DNS hierarchy in Docker across three isolated subnets that are joined only by two no-NAT routers. First we run plain DNS, then attack it from an on-path position, then add defences one at a time (DNSSEC, and finally an encrypted last mile) and see which attack each one actually stops.

Addresses. client `172.16.1.10`, webserver `172.16.1.20`, rogue-web `172.16.1.30`; resolver `172.16.2.10`, attacker `172.16.2.100` (phases 2 to 5) or `172.16.1.100` (phases 6 to 7); root-ns `172.16.3.10`, tld-ns `172.16.3.20`, domain-ns `172.16.3.30`; edge-router `172.16.1.2`/`172.16.2.2`, core-router `172.16.2.3`/`172.16.3.3`.

Shared code (`common/`). `base/` has the Phase 1 hierarchy and the BIND 9.20 image, `signing/` does the DNSSEC signing, and `interceptor/` is the C/libpcap attack tool. Every `phaseN/` folder is self-contained and reuses these.

We ran everything on Colima (Docker 29, Debian trixie containers, Apple Silicon). Every result below is real output from that phase's `scripts/`, and the full terminal logs and packet captures are saved under each `phaseN/evidence/`. Packet captures are taken with `tcpdump` and decoded with `tshark`, both used purely as inspection tools. All attack programs (the interceptor and the Phase 5 zone-walker) are C with libpcap, capturing and injecting with `pcap_inject`, as the brief requires; no Scapy, dnspython, or dig-based attack code is used.

Phase 1: Building the DNS Hierarchy

Design choices

Isolation comes from the topology, not from firewall rules. Each subnet is a Docker bridge with `internal: true`, so there is no NAT and no Internet. A packet only crosses subnets through a router that sits on both bridges, and the forwarding happens inside that router container, so we never have to fight Docker's own bridge isolation. Nothing else forwards.

Each node installs its own static routes from `STATIC_ROUTES` in a small entrypoint, which only needs `NET_ADMIN`: client reaches the resolver net via edge-router, the resolver reaches the auth net via core-router and the client net via edge-router, and each nameserver reaches the resolver net via core-router. The routers need no extra routes since both of their subnets are directly connected.

Addresses stay real end to end: root hints and glue point at the nameservers' `172.16.3.x` addresses. The resolver is recursive with DNSSEC validation off.

Report answers

Resolution, fetch, and the isolation check:

```
$ docker exec client dig +short example.com
172.16.1.20
$ docker exec client dig +short www.example.com
172.16.1.20
$ docker exec client curl -s http://example.com | grep -i welcome
  <h1>Welcome to example.com</h1>
$ docker exec client dig +time=2 +tries=1 @172.16.3.10 example.com
```

```
;; no servers could be reached        # auth subnet is not directly reachable
```

The referral chain, from a `tshark` capture on core-router of one `example.com/A` resolution (deduped; the responses that carry it):

```
172.16.3.10 -> 172.16.2.10  resp 0x0e4b  com NS a.gtld-servers             # root: referral to com
172.16.3.10 -> 172.16.2.10  resp 0x75c9  a.gtld-servers A 172.16.3.20      # root: glue for com NS
172.16.3.20 -> 172.16.2.10  resp 0x0614  example.com NS ns.example.com A 172.16.3.30   # com: referral
172.16.3.30 -> 172.16.2.10  resp 0xa8cc  example.com A 172.16.1.20         # domain-ns: final answer
```

Everything on the wire is cleartext. Each line shows the server, the 16-bit transaction ID (`resp 0x...`) that ties a reply to its query, the QNAME, the record type, and the data: the referral NS names with their glue addresses, then `example.com A 172.16.1.20`. The root is also queried a few extra times to resolve the nameserver names themselves (they live in their own pseudo-TLDs); resolution still finishes from the glue.

How the resolver's query reaches the root:

```
$ docker exec resolver ip route get 172.16.3.10
172.16.3.10 via 172.16.2.3 dev eth0 src 172.16.2.10
```

The resolver has no leg on the auth subnet, so it uses the static route `172.16.3.0/24 via 172.16.2.3` and hands the packet to core-router, which forwards it out of its `172.16.3.3` interface to root-ns.

Because the whole exchange is unauthenticated cleartext, any node on the path can read a query and forge a reply by copying its QNAME, ID, and ports. That is exactly Phase 2.

Phase 2: On-Path Response Forgery

Design choices

The diversion is a single route on the router. We add `ip route replace 172.16.2.10/32 via 172.16.2.100` on core-router, so the resolver's inbound responses go to the attacker first. The resolver and nameservers keep their Phase 1 config.

The attacker does not forward. It has `NET_RAW` (plus `NET_ADMIN`) and runs the C tool in `common/interceptor/dns_intercept.c`. Since the kernel would drop the diverted frames (wrong destination IP, and `ip_forward=0`), we use `pcap_open_live` to capture and `pcap_inject` to relay them, which is the capture-and-inject path the brief asks for.

The forge is selective. A BPF filter picks only the diverted DNS responses (`ether dst <us> and udp and src port 53 and dst host 172.16.2.10`), and never our own injected frames, so there is no loop. We rewrite the A record only when the question is exactly `example.com` with type A. IP and UDP checksums are recomputed and the packet length does not change.

Evidence

Terminal, relay then forge:

```
relay: example.com      -> 172.16.1.20
forge: example.com      -> 172.16.1.30
forge: www.example.com  -> 172.16.1.20
forge: curl example.com -> <h1>:: HACKED ::</h1>
```

The forgery on the wire: `tshark` excerpts of the same response captured at its source (domain-ns) and at its destination (resolver). Same transaction ID `0x8d38`, same QNAME, type, and ports; only the A rdata is rewritten in flight:

```
# tshark on domain-ns (what the authoritative server actually sent):
172.16.3.30 -> 172.16.2.10  DNS 159  Standard query response 0x8d38 A example.com A 172.16.1.20
# tshark on resolver (what arrived after the attacker):
172.16.3.30 -> 172.16.2.10  DNS 159  Standard query response 0x8d38 A example.com A 172.16.1.30
```

The source address is still `172.16.3.30` because the attacker relays the packet with the server's IP intact; only the answer changed, which is exactly why the resolver accepts it.

The resolver caches the poison: a second lookup is served from cache with a TTL that has already decayed, so it is not re-fetched:

```
first dig TTL : 3599
after 3s TTL  : 3596
```

Report answers

Plain DNS has no origin authentication. A resolver accepts any reply that matches the query's QNAME, type, 16-bit ID, and UDP port. An off-path spoofer has to guess the ID and port and win a race against the real server. Our on-path attacker just reads them from the genuine packet and rewrites it, so there is nothing to guess and no race, and the attack is deterministic.

Non-target names are safe because the tool only rewrites answers when the QNAME is exactly `example.com` and the type is A. `www.example.com` is a different QNAME, so it falls through the relay path and is injected byte for byte, which is why it still returns `172.16.1.20`.

Phase 3: DNSSEC Chain of Trust

Design choices

We sign with ECDSAP256SHA256 (algorithm 13). ECDSA P-256 produces compact keys and signatures and is widely supported, and validation is fast. Compared with RSA it may have less extensive legacy compatibility; Ed25519 is a newer alternative. The compact signatures keep the signed zones small, which suits this lab.

The chain is built child first (`common/signing/sign-all.sh`): sign `example.com`, put its KSK's DS into `com`, sign `com`, put its DS into the root, then sign the root. The resolver anchors on the root KSK only (`trust-anchor.conf`) and turns on `dnssec-validation yes`. Root hints are unchanged from Phase 1. The attacker and its Phase 2 binary are untouched.

No private key is in the repository. Keys and signed zones are build artifacts that the signer regenerates, and they are git-ignored.

Verify (healthy signed hierarchy)

All queries run on the `client` container against the resolver at `172.16.2.10`:

```
$ docker exec client dig +dnssec @172.16.2.10 example.com
;; flags: qr rd ra ad; ...                              # authenticated-data flag set
example.com.  3600  IN  RRSIG  A 13 2 3600 ...          # signature present
$ docker exec client delv -a trust-anchor.conf @172.16.2.10 example.com
; fully validated
example.com.  3600  IN  A  172.16.1.20
```

The signed records travel in cleartext. A `tshark` capture on core-router of one `dig +dnssec` resolution counts the DNSSEC records on the wire:

```
2  DS        12  DNSKEY        21  RRSIG     # all readable, none encrypted
```

Attack (Phase 2 forgery re-run, unchanged)

```
$ docker exec client dig @172.16.2.10 example.com
;; ->>HEADER<<- status: SERVFAIL                         # forged answer rejected
# resolver log:
RRSIG failed to verify resolving 'example.com/A/IN': 172.16.3.30#53
# the client fetch then fails to resolve at all (it gets neither the real nor the rogue page):
$ docker exec client curl -v http://example.com
* Could not resolve host: example.com
```

```
curl: (6) Could not resolve host: example.com
```

A `tshark` capture on the resolver shows why validation fails. The attacker rewrote the A to `172.16.1.30` but left the original RRSIG in place (it has no key to re-sign), so the signature no longer matches the answer it covers:

```
172.16.3.30 -> 172.16.2.10  resp 0x24f3  example.com A 172.16.1.30  + RRSIG A (invalid for the modified
    ↪ A RRset)
```

The RRSIG is right there on the wire next to the forged A; the resolver checks it against the A RRset, finds no match, and returns SERVFAIL. In Phase 2 the identical packet was accepted because nothing checked the signature.

So the forgery does not poison the client; it denies it. The resolver returns SERVFAIL rather than an address, so `curl` cannot resolve the name and no page is fetched (contrast Phase 2, where the same packet poisoned the client to the rogue).

Report answers

The attacker injects the same forged A, but it cannot produce a matching RRSIG because it has no zone private key. The validating resolver walks the chain (root KSK, DS(com), com KSK, DS(example.com), example.com ZSK) and checks the signature over the A RRset. It finds no valid signature, so it cannot call the name secure and returns SERVFAIL instead of the poison. In Phase 2 the same packet was accepted because nothing was checked.

On the wire the exchange is still cleartext, so the on-path attacker can read the QNAME, the IDs, and the answers just like in Phase 1, now with the RRSIG, DNSKEY, and DS records visible too. So DNSSEC gives origin authentication and integrity of the DNS data, but not confidentiality, and it does not protect the last hop to a stub that does not validate. That last hop is Phase 6.

Private keys do not belong in the repo. We commit only the signing scripts and the public trust anchor; the KSK and ZSK private keys are regenerated each run.

Phase 4: Signature Stripping and Downgrade

Design choices

Forging a signed answer failed in Phase 3, so here the attacker tries to make the signed zone look unsigned and get the plain Phase 2 forgery accepted again. The interceptor gets a third mode, `strip` (see `strip_rebuild()`): for `example.com` it rebuilds the message with the forged A and every signature that protects the A, DS, or DNSKEY removed. That strips the RRSIG from the `example.com` answer and, for the fuller downgrade, the DS and its RRSIG from the `com` referral and DS responses. The attacker holds no private keys.

The two runs differ by one line in the signer (`NODS` in `sign-all.sh`), which tells `com` whether to publish the DS for `example.com`.

Run A: com publishes the DS (secure delegation)

```
# baseline (no attacker): example.com validates
;; flags: qr rd ra ad                                  # authenticated
# strip attack:
$ docker exec client dig example.com
;; ->>HEADER<<- status: SERVFAIL
# resolver log:
broken trust chain resolving 'example.com/A/IN': 172.16.3.30#53
```

The DS strip is visible on the wire: during the attack the `com` response for `example.com/DS` is cut down to just the question and OPT, with the DS record gone:

```
172.16.3.20 -> 172.16.2.10  DNS 110  resp 0xa08f  DS example.com OPT   # DS stripped
```

Run B: com publishes no DS (insecure delegation)

```
# baseline (no attacker): example.com resolves but is NOT authenticated
;; flags: qr rd ra                                    # no ad flag, insecure
# same strip attack now lands:
$ docker exec client dig +short example.com
172.16.1.30                                          # rogue "HACKED" page
```

The forged answer the resolver accepts has the A rewritten and no signature over it (the capture shows Answer RRs: 1, a lone `example.com A 172.16.1.30`, the RRSIG gone):

```
172.16.3.30 -> 172.16.2.10  resp 0xa9ff  example.com A 172.16.1.30  (Answer RRs: 1, no RRSIG)
```

Report answers

In Run A the signatures, and even the DS, can be stripped on the wire, but the attacker still cannot make the zone look unsigned. "Unsigned" is not the absence of signatures, it is a signed statement by the parent that the child has no DS. `com` still says, with a signature the attacker cannot forge, that `example.com` is signed, so when the DS is stripped the resolver cannot complete the chain and returns SERVFAIL ("broken trust chain") rather than trust the unsigned answer.

In Run B the single change is dropping `example.com`'s DS from `com`. That flips the parent's signed statement to "this child is insecure", `com` now carries a signed NSEC proving no DS exists, and the resolver believes it because it is authenticated. From then on `example.com` is plain DNS and the Phase 2 forgery works again.

So downgrade resistance does not come from the child being signed. It comes from the secure delegation: the DS in the parent, chained to the trust anchor. The exact operator mistake that forfeits it is signing a zone but never publishing (or later dropping) its DS in the parent.

Phase 5: Zone Walking

Design choices

A signed zone must prove a name does not exist, and that proof is signed ahead of time, before anyone knows which name will be asked. It cannot be about the queried name, so it reveals something about the names that do exist. With NSEC, what it reveals is the next existing name. Our `example.com` zone has nine owner names: the apex `example.com` and `www` are advertised, `ns` is the nameserver name (public through the NS record and glue), and the other six (`backup, dev, git, mail, staging, vpn`) are internal names that appear nowhere public. The walk should recover all of them, and in particular those six intentionally hidden ones.

The walker is C with libpcap (`walker/zonewalk.c`): it builds each DNS query by hand, injects it with `pcap_inject`, and reads the reply back off the wire with libpcap, as the brief requires for the zone-walker. `dig` is used only to inspect the signed denial.

Verify and attack

A nonexistent name is denied with signed NSEC records that each point from one real name to the next, which leaks a real neighbour. The authority section for `nope.example.com` carries two NSEC records, each followed by the RRSIG that signs it:

```
$ docker exec resolver dig +dnssec +norec nope.example.com @172.16.3.30
mail.example.com.  NSEC   ns.example.com.      A RRSIG NSEC
mail.example.com.  RRSIG  NSEC 13 3 3600 20261122163900 20261023163900 27726 example.com. <sig>
example.com.       NSEC   backup.example.com.  A NS SOA RRSIG NSEC DNSKEY
example.com.       RRSIG  NSEC 13 2 3600 20261122163900 20261023163900 27726 example.com. <sig>
```

Each NSEC is covered by a real `RRSIG NSEC`, so the denial is authenticated. The first NSEC (`mail -> ns`) proves `nope` does not exist, because `nope` sorts between `mail` and `ns`; the apex NSEC proves there is no `*.example.com` wildcard that could have synthesised an answer. Together they are the complete, signed denial, and both leak a real neighbour, which is exactly what the walk exploits.

Following that chain from the apex recovers the whole zone, one query per name:

```
$ docker exec walker /opt/walker/zonewalk -s 172.16.3.30 -g 172.16.2.3 -z example.com
[zonewalk] walking example.com via NSEC at 172.16.3.30 (libpcap capture+inject)
  NSEC example.com         -> backup.example.com
  NSEC backup.example.com  -> dev.example.com
  NSEC dev.example.com     -> git.example.com
  NSEC git.example.com     -> mail.example.com
  NSEC mail.example.com    -> ns.example.com
  NSEC ns.example.com      -> staging.example.com
  NSEC staging.example.com -> vpn.example.com
  NSEC vpn.example.com     -> www.example.com
  NSEC www.example.com     -> example.com
NSEC queries sent : 9
names recovered   : 8
  backup 10.0.0.5  dev 10.0.0.6  git 10.0.0.7  mail 10.0.0.8
  ns 172.16.3.30   staging 10.0.0.9  vpn 10.0.0.10  www 172.16.1.20
```

Nine NSEC queries recover all eight non-apex names: the six intentionally hidden ones (`backup, dev, git, mail , staging, vpn`) plus the nameserver name `ns` and the advertised `www`. That is within the (owner names + 2) budget for a zone of nine owner names, and the six hidden names, which appear nowhere public, are fully exposed. The queries really are injected by the walker, as a capture at domain-ns shows (source `172.16.2.100`, each NSEC reply carrying the next owner):

```
172.16.2.100 -> 172.16.3.30  query 0xc0fe  NSEC example.com
172.16.3.30 -> 172.16.2.100  resp  0xc0fe  NSEC example.com -> backup.example.com RRSIG ...
```

Fix

Re-signing with NSEC3 (`NSEC3=1 sh sign.sh`) hashes the owner names before chaining them. The same C walker now finds no NSEC to follow and recovers nothing, while a nonexistent name still gets an authenticated denial, now via NSEC3:

```
$ docker exec walker /opt/walker/zonewalk ...
  no NSEC returned for example.com : chain not exposed (NSEC3 or not signed)
names recovered   : 0
# the denial is still authenticated: an NSEC3 record over hashed names, plus its RRSIG
TNEQ4S3S...AHV5MC4ATJS.example.com.  NSEC3  1 0 0 10004BC7...  (hashed owner)
TNEQ4S3S...AHV5MC4ATJS.example.com.  RRSIG  NSEC3 13 3 3600 ...  example.com.   (signed)
```

The walker is unchanged, so the recovery of zero names is purely the effect of NSEC3, and the nonexistent name still gets a signed NSEC3 denial, not an unsigned or missing response.

Report answers

NSEC made the walk possible. Plain DNS answers NXDOMAIN on demand with nothing about other names; DNSSEC cannot sign on demand without an online key, so denial is precomputed as one NSEC per gap between adjacent names, and those records chain every name to the next. NSEC3 hashing stops the straightforward online walk, but it adds hashing and validation work and operational complexity (not inherently many more denial records than NSEC), and it is not secrecy: an enumerator can still count the records to learn the zone size and take the hashes offline for a dictionary attack. The real fix for secret names is to keep them out of public DNS.

Phase 6: Last-Mile Response Forgery

Design choices

The resolver validates, but the hop from resolver to client carries no proof of it: a stub trusts whatever appears to come from its resolver. So the attacker moves onto the client subnet (`172.16.1.100`) and a route on `edge-router` (not ARP spoofing) sends a client's inbound replies through it. Upstream is the Phase 3 signed, validating

resolver, untouched. Two victims: `client-a`, a plain stub using `curl` and the OS resolver, and `client-b`, which validates itself with `delv` against its own root anchor.

Attack

```
# Client A (plain stub):
client-a example.com      -> 172.16.1.30   (rogue "HACKED" page)
client-a www.example.com  -> 172.16.1.20   (untargeted, untouched)

# Client B (validating), same forgery on the wire:
plain dig on client-b -> 172.16.1.30                        # would be fooled
delv on client-b      -> RRSIG failed to verify resolving 'example.com/A/IN': 172.16.2.10#53
```

A capture on the client subnet shows the forged reply arriving with the `ad` (authenticated-data) bit set, even though the attacker produced it. `tshark` field extraction of the `example.com` A response on its way to the client:

```
# fields: ip.src        dns.a          dns.flags.authenticated
          172.16.2.10   172.16.1.30    1          # AD bit set on a forged answer
```

The attacker sets (or leaves) the AD bit at will on the last hop; the stub has no signature to check, so it believes it. This is the capture behind the Client A result above.

Report answers

DNSSEC validates upstream but does not stop this, because validation happens at the resolver while trust is placed by the stub. The resolver validated the real answer and set the `ad` flag, but `ad` is one bit in a cleartext packet on the last hop with nothing signing it, so an on-path attacker rewrites the answer and sets `ad` for free, and the stub believes it. Client B does not rely on `ad`: it runs its own validation against a trust anchor it holds, so it needs real signatures, which the attacker cannot produce, and the forged A is rejected. A plain `dig` on the same machine is fooled, so the difference is the local validation, not the network. Protection comes from validating where trust is placed, or from authenticating the hop itself, which is Phase 7.

Phase 7: Encrypting the Last Mile (DoT / DoH)

Design choices

We stand up a private CA (`pki/gen-ca.sh`): an RSA-2048 root CA issues the resolver a certificate whose CN and SAN are `resolver.cs6008.local`, with a DNS SAN only and no IP SAN. The CA goes into the client's system trust store with `update-ca-certificates`, so there is no `-k` or insecure bypass. The client reaches the resolver by name, never by IP (`extra_hosts: resolver.cs6008.local -> 172.16.2.10`), so the certificate is checked against the hostname. On the resolver we add `tls local-tls { cert-file; key-file; }` with `listen-on port 853 tls` for DoT and `listen-on port 443 tls ... http` for DoH. Plain port 53 stays up so the downgrade attempt has something to fall back to. Phase 3 DNSSEC validation still runs upstream.

Verify

```
# DoT:  ;; TLS session (TLS1.3)-(ECDHE-X25519)-(RSA-PSS-RSAE-SHA256)-(AES-256-GCM)
#       ;; status: NOERROR  example.com A 172.16.1.20   From 172.16.2.10@853(TLS)
# DoH:  ;; HTTP session (HTTP/2-POST)-(resolver.cs6008.local/dns-query)-(status: 200)  @443(HTTPS)
# enforced: wrong hostname -> handshake failed (certificate error); missing CA -> fails
```

A `tshark` capture on edge-router during a DoT lookup shows a TLS 1.3 handshake on 853 and no DNS dissected anywhere (the query is inside the encrypted session):

```
172.16.1.10 -> 172.16.2.10  TLSv1.3  385   Client Hello
172.16.2.10 -> 172.16.1.10  TLSv1.3  1536  Server Hello, Change Cipher Spec, Application Data
# tshark display filter "dns" on the same capture: 0 packets   # no cleartext DNS on :53
```

Attack: three attempts against two client policies

`strict` uses DoT only and never falls back; `lax` prefers DoT but falls back to cleartext :53 when the encrypted transport cannot be reached (a bad cert is an attack, not an unavailable transport, so it does not trigger fallback). Measured:

```
 attempt                          | strict (refuses cleartext) | lax (permits cleartext)
----------------------------------+----------------------------+------------------------
 (i)  alter answer in transit     | no  (TLS integrity)        | no  (TLS integrity)
 (ii) rogue-cert impersonation    | no  (cert not trusted)     | no  (cert not trusted)
 (iii) block transport, downgrade| no, fails closed (DoS)     | YES, forged to rogue
```

(i) The DNS is inside TLS, so the cleartext forge tool sees nothing and a capture of 853 has zero readable `example.com` strings; both clients get the real answer. (ii) A rogue server presents a cert for the right name but self-signed, not from the lab CA; both clients reject it (`handshake failed, certificate error`). (iii) Blocking 853/443 and forging :53 downgrades only the lax client, which falls back to cleartext and takes `172.16.1.30`; the strict client fails closed.

Report answers

Five of six combinations fail. The one that lands, (iii) against lax, does not beat the cryptography; it beats a client policy that still allows cleartext. We deployed both DoT and DoH and lead with DoT: DoT on its own port 853 is simple to audit and just as simple to spot and block (as demonstrated in attempt (iii)), while DoH on 443 blends into HTTPS and resists blocking at the cost of being harder to audit and heavier. An encrypted, authenticated last mile protects the client-resolver hop only. It does nothing for the resolver-to-authoritative path (still plain :53, DNSSEC gives integrity not confidentiality there), it does not hide which resolver you use, and it does not stop a malicious or misconfigured resolver from lying over a valid TLS session: the channel is authenticated, the answer is not. Only client-side DNSSEC validation catches that.

Appendix: Generative AI usage

AI assistance (Claude Code) was used throughout. It wrote the Compose files, the BIND configs and zone files, the routing scripts, the DNSSEC signing scripts, the C/libpcap interceptor, the C/libpcap zone-walker, the Phase 7 PKI and DoT/DoH setup, and a first draft of this report.

Prompts used (the actual prompts that drove the build, lightly cleaned up):

- "Build the Phase 1 hierarchy: three isolated Docker subnets joined by two no-NAT routers, BIND 9.20, root/com/example.com zones with glue, a recursive resolver."
- "Write a C/libpcap on-path interceptor that relays diverted DNS and, in forge mode, rewrites only the A record for an exact target name; recompute IP/UDP checksums."
- "Write the DNSSEC signing scripts: sign child-first, chain DS parent to child, emit a root trust anchor; add an NSEC3 option and an option to omit a child's DS."
- "Add a strip mode to the interceptor that removes the RRSIG/DNSKEY/DS so a signed answer looks unsigned; show it fails when the parent has a DS and succeeds when it does not."
- "Write the Phase 5 zone-walker in C with libpcap (build and inject the query, capture the reply); follow the NSEC chain from the apex. No dig, Scapy, or dnspython."
- "Stand up a private CA and DoT/DoH on the resolver, reach it by name with cert validation, then run the three last-mile attacks against a strict and a lax client."

Verification: nothing was taken on trust. Every phase was deployed on Docker and checked against the brief's pass/fail criteria, with terminal output and `tshark` captures saved under each `phaseN/evidence/` (for example Phase 3's `delv` "fully validated" then SERVFAIL, Phase 4's broken-chain versus rogue outcomes, Phase 5's full recovery then none under NSEC3, and Phase 7's six attack results).